Hybrid Deepfake Detector - Comprehensive Project Report

Hybrid Deepfake Detector - Comprehensive Project Report

1. Executive Summary

This project implements a hybrid deepfake detection system that analyzes videos for signs of manipulation using multiple, complementary cues:

• Convolutional Neural Network (CNN) on face crops (visual artifacts)

• Remote Photoplethysmography (rPPG) from subtle facial color changes (physiological signal)

• Temporal noise/artifact analysis (frame-difference variance)

• Explainable, user-facing web interface with detailed metrics, visualizations, and PDF reporting

Key deliverables:

• Command-line detector (hybrid_deepfake_detector.py)

• Streamlit web app (app.py) with upload, analysis, dashboards, and PDF export

• One-click launcher for Windows (run_app.bat)

• Training & evaluation utilities (prepare_data.py, quick_train.py, train_model.py, evaluate_model.py)

---

2. Repository Structure

---

3. System Overview & Architecture

3.1 High-Level Flow

1. Load video → extract frames

2. Face detection (MediaPipe) → face crops

3. CNN inference on sampled face frames → per-frame fake probabilities

4. rPPG estimation from facial green channel over time → SNR, BPM

5. Noise/artifact score from temporal frame diffs (variance)

6. Fusion of signals (adaptive weighting) → final deepfake score

7. Thresholding → decision (REAL/FAKE)

8. Reporting → JSON + Streamlit dashboard + PDF export

3.2 Core Components

• Face detection: mediapipe.solutions.face_detection

• CNN backbone: ResNet18 (ImageNet topology; classifier adapted for 2 classes)

• rPPG: Welch PSD peak on green-channel mean across face crops

• Noise metric: Variance of consecutive frame differences (normalized)

• Fusion logic: Adaptive weights + robust aggregation (trimmed-mean + median) over CNN frame probabilities

---

4. Algorithms & Methods

4.1 CNN (Visual Artifact Detection)

• Backbone: torchvision.models.resnet18(weights=None), final FC → 2 logits (real/fake)

• Pretrained weights file: models/resnet18_ffpp.pth (trained on FaceForensics++-style data)

• Inference:

• Sample ~64 frames uniformly

• Face crop per frame (first face)

• Preprocess: resize to 224×224, normalize (ImageNet mean/std)

• Batched inference with optional AMP on CUDA

• Robust aggregation: combine per-frame probabilities by trimmed-mean (10% each side) + median (70/30 mix)

4.2 rPPG (Physiological Cue)

• Signal: Mean of green channel on face crop per frame

• PSD via Welch; peak frequency → BPM; SNR as peak/mean ratio

• Rule: Low SNR (<5) suggests unstable physiological signal → contributes to fake

4.3 Temporal Noise / Artifact

• Compute np.var(np.diff(frames)) / 1e6 (safe-guarded) as normalized motion/noise proxy

• Higher noise/artifacts can correlate with synthesis artifacts

4.4 Decision Fusion

• Adaptive weights:

• Default: {cnn: 0.6, rppg: 0.3, noise: 0.1}

• If rPPG skipped/unavailable: {cnn: 0.8, rppg: 0.0, noise: 0.2}

• If few CNN frames (<5): down-weight CNN, re-normalize with noise

• Final score clamped to [0, 1]

• Decision threshold: 0.5 for “FAKE”; otherwise “REAL”

Rationale: Multiple cues reduce false positives/negatives; robust aggregation mitigates outlier frames.

---

5. Web Application (Streamlit)

5.1 Features

• Upload video (MP4/AVI/MOV/MKV)

• Progress feedback (extract → inference → fusion)

• Prominent REAL/FAKE banner + confidence bars

• Metrics cards: CNN prob, rPPG SNR/BPM, noise score, final score

• Plotly charts (bar, gauge)

• Technical notes expander

• “Download Detailed Report” → PDF (ReportLab)

5.2 PDF Report Contents

• Title, timestamp, filename

• Decision and confidence

• Summary table (Final Score, CNN %, rPPG SNR/BPM, Noise)

• Detailed analysis narratives (CNN/rPPG/Noise)

• Methodology section

---

6. Running the Project

6.1 One-Click (Windows)

• Double-click run_app.bat.

• Creates .venv (prefers Python 3.11), installs deps, starts app.

• Browser opens at http://localhost:8501 (or indicated port)

6.2 Manual

6.3 CLI Detector

Creates out/report.json with detailed metrics.

---

7. Training & Improving Accuracy

Use the included scripts to prepare data, train, and evaluate.

7.1 Data Preparation

Output structure:

7.2 Quick Fine-Tuning

Saves models/improved_model.pth and models/final_trained_model.pth.

7.3 Advanced Training

Model selection supported: resnet18, resnet50, efficientnet_b0, vit_b_16.

7.4 Evaluation & Threshold Tuning

Outputs: confusion matrix, ROC, probability distributions, optimal threshold suggestion.

7.5 Deploying Improved Model

Replace the UI model file:

Restart Streamlit.

---

8. Design Decisions & Trade-offs

• ResNet18: light and fast; easy to swap to larger backbones

• MediaPipe face detection: robust and simple; single-face assumption for speed

• rPPG: boosts reliability on real videos; may be skipped on low-quality or fast-mode

• Fusion with robust aggregation: stabilizes per-frame volatility

• Streamlit: fastest route to interactive UI + built-in file handling and charts

---

9. Performance & Accuracy Notes

• Accuracy depends on training data. For best results:

• Use diverse datasets (FaceForensics++, Celeb-DF, DFDC)

• Align faces and ensure sufficient face area

• Sample ~64 frames; longer videos improve rPPG stability

• Tune threshold on a validation set

• Speed optimizations:

• Batched CNN inference

• cuDNN benchmark enabled

• AMP on CUDA during inference

---

10. Limitations

• Single visible face assumed (first detection used)

• rPPG reliability drops under extreme motion/lighting

• ResNet18 may miss sophisticated manipulations; consider stronger backbones or ensembles

• Audio/phoneme lip-sync analysis is not implemented (notes placeholder only)

---

11. Future Work

• Ensembling (ResNet50 + EfficientNet + ViT)

• Temporal models over per-frame features (GRU/Temporal CNN)

• Lip-sync analysis (audio features + mouth landmarks)

• Face alignment and quality filters before inference

• Calibration (temperature scaling) for better probability estimates

---

12. Troubleshooting

• “No faces detected → CNN skipped”: ensure the face is visible and large enough

• rPPG SNR is None: too few frames or unstable detection; try a longer/clearer clip

• Wrong classification: increase sampled frames; tune threshold using evaluate_model.py; consider retraining

• Streamlit not starting: verify Python and pip install -r requirements.txt

---

13. Dependencies

---

14. Security & Ethics

• Deepfake detection can produce false positives/negatives; decisions should be used with caution.

• Always disclose uncertainty to users; avoid sole reliance for critical decisions.

• Ensure privacy: process videos locally; avoid uploading sensitive data to third-party services.

---

15. Credits

• Backbone architectures from torchvision

• MediaPipe Face Detection by Google

• Datasets inspiration: FaceForensics++, Celeb-DF, DFDC

---

16. Change Log (Key Enhancements)

• Robust aggregation (trimmed-mean + median) for CNN per-frame probabilities

• Adaptive fusion weighting; stricter fake threshold (0.5)

• Batched inference + AMP + cuDNN benchmark

• Streamlit UI with enhanced clarity, PDF reporting, and one■click launcher

---

17. How to Cite

If you use this project in academic or industrial work, please cite as:

“Hybrid Deepfake Detector (Mini Project), 2025. A multi-cue deepfake detection system combining CNN-based visual analysis, rPPG estimation, and temporal noise metrics with a user-facing web interface.”